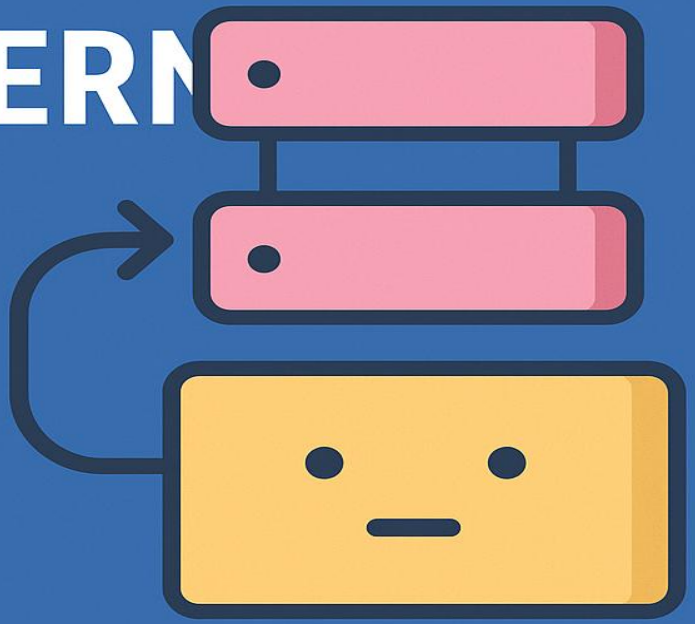
DECORATOR
DESIGN PATTERN



Creational (5)

- [x] Factory
- [ ] Abstract Factory
- [x] Singleton
- [ ] Prototype
- [ ] Builder

Structural (7)

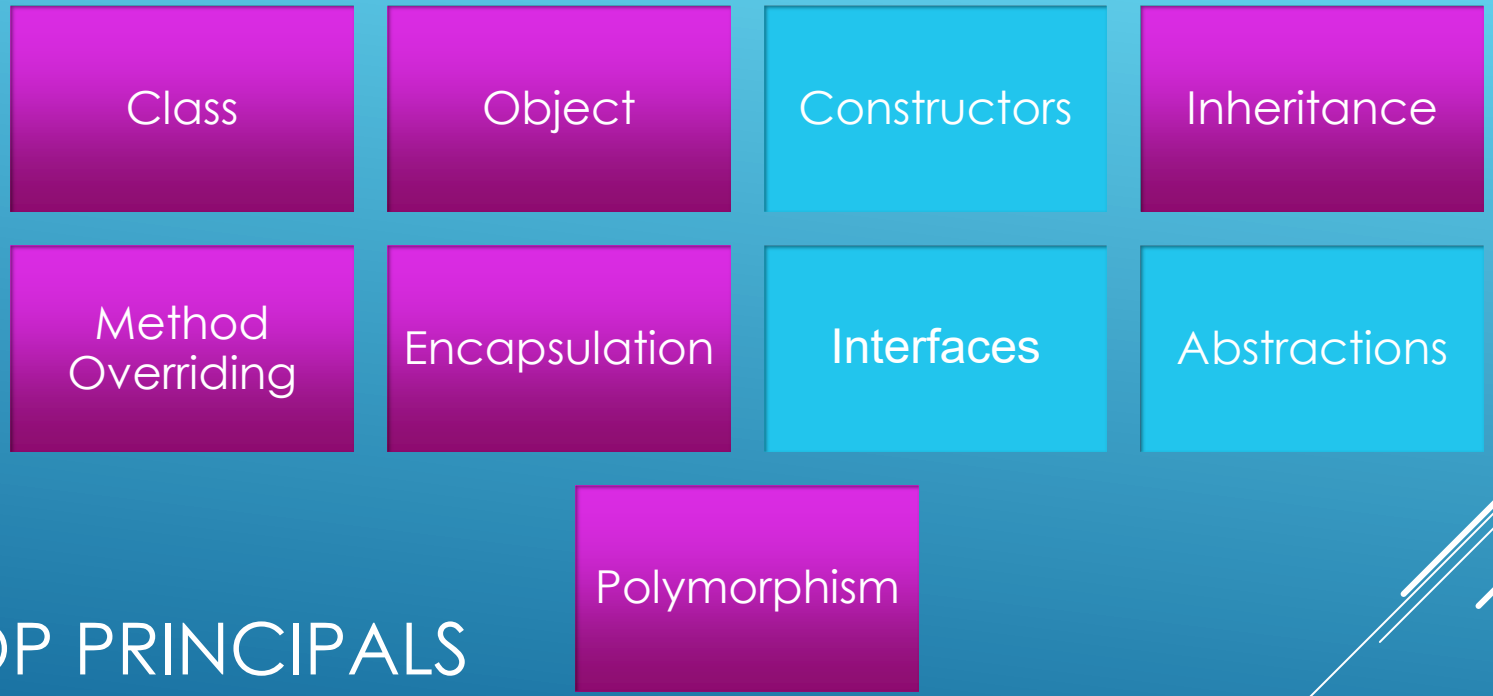
- [x] Adapter
- [ ] Bridge
- [ ] Composite
- [ ] 

Decorator
- [x] Facade
- [ ] Flyweight
- [x] Proxy

Behavioral (11)

- [ ] Chain Of Responsibility
- [ ] Command
- [ ] Interpreter
- [ ] Iterator
- [ ] Mediator
- [ ] Memento
- [ ] Observer
- [ ] State
- [ ] Strategy
- [ ] Template
- [ ] Visitor

OOP PRINCIPALS



HOW WOULD YOU MODEL DIFFERENT TYPES OF COFFEE?



- One class many properties?
- Many classes?













HTTPS://HOPPERSUNLIMITED.COM.AU/VARIETIES/



COFFEE SHOP – POSSIBLE ADD-ONS



WHAT ARE THE COMBINATIONS

Number of Properties	Combinations
3	8
5	252
10	1024
15	32,768

Combinations for n = 3
NULL
A
B
C
A,B
B,C
A,C
A,B,C

- **Add-ons / Ingredients**
- Milk
- Sugar
- Honey
- Cream
- Whipped Cream
- Vanilla Flavor
- Caramel Flavor
- Hazelnut Flavor
- Chocolate Syrup
- Cinnamon Sprinkle
- Ice
- Extra Espresso Shot
- Soy Milk
- Almond Milk
- Oat Milk

Size / Strength

- Small Size
- Medium Size
- Large Size
- Decaf
- Extra Strong Brew

- **Serving Style**
- Takeaway Cup
- Reusable Cup
- Glass Mug

- **Seasonal / Special**
- Pumpkin Spice
- Peppermint Flavor
- Festive Toppings

INTRODUCTION



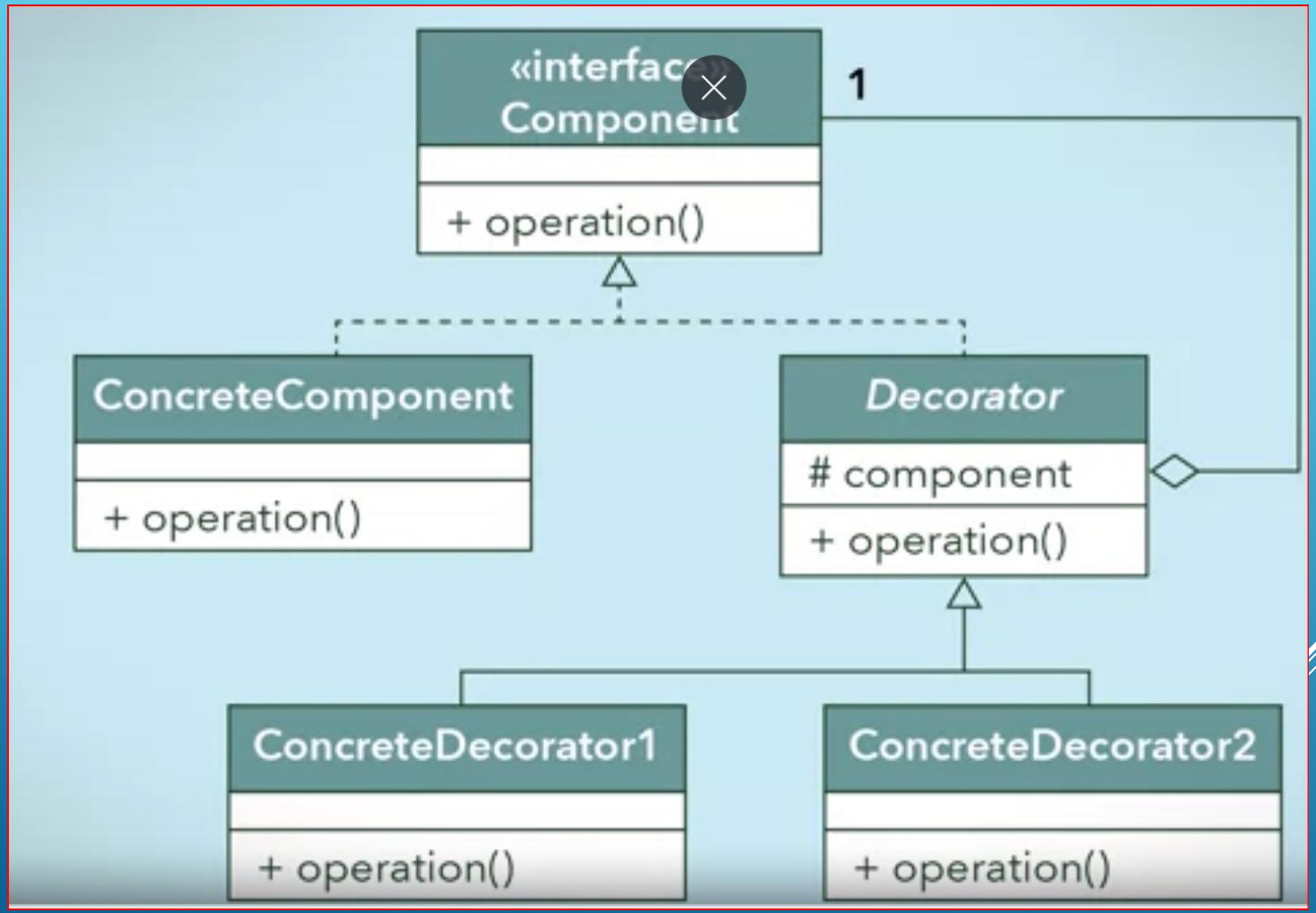
Allows behaviour / property to be added to an individual object without impacting the behaviour of the class.

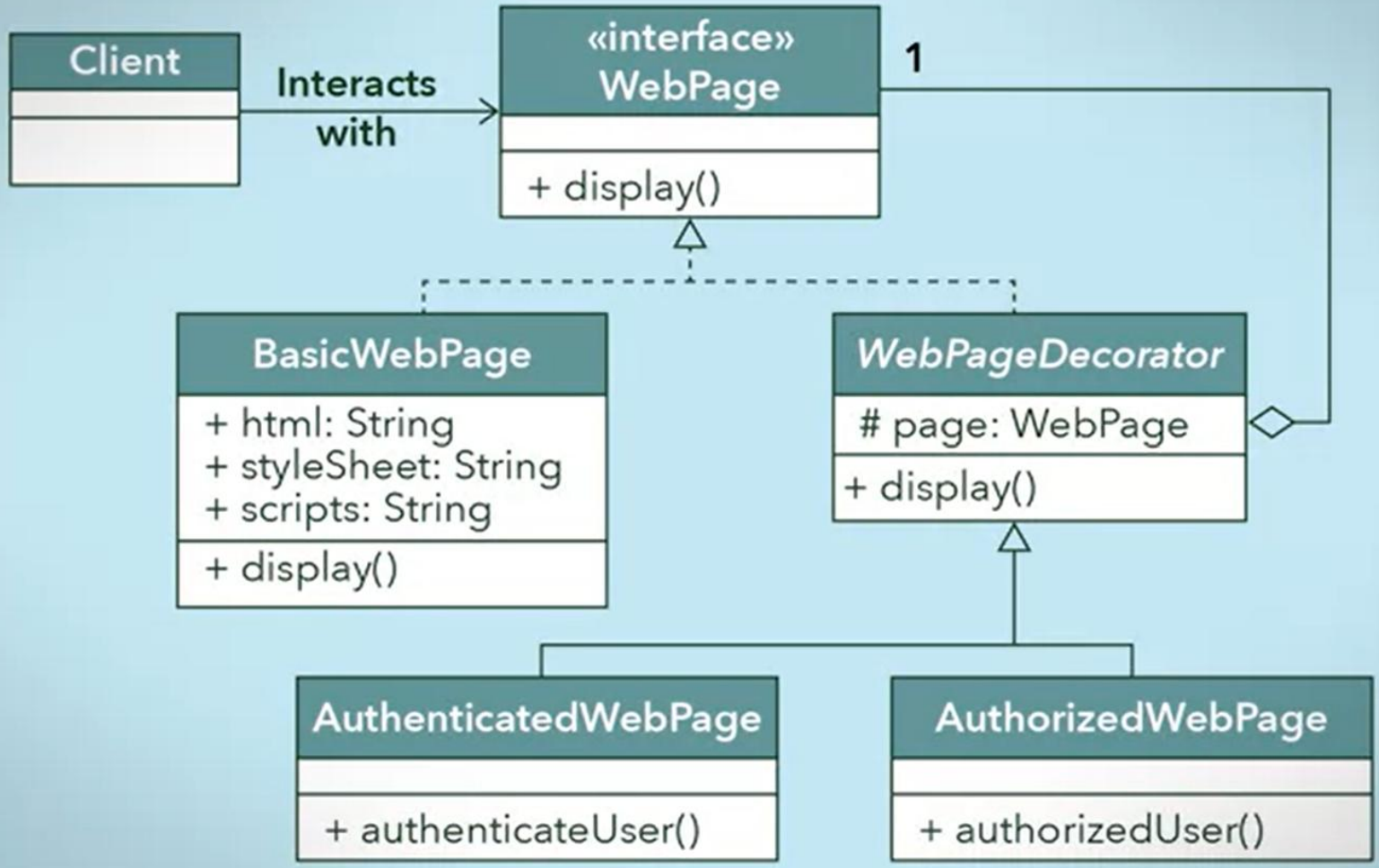


This can be added statistically or dynamically.



Alternative for sub classing and avoid many number of classes or many number of attributes


«interface»
Component
+ operation()
1
ConcreteComponent
+ operation()
Decorator
component
+ operation()
ConcreteDecorator1
+ operation()
ConcreteDecorator2
+ operation()


Client
Interacts with
«interface»
WebPage
1
+ display()
BasicWebPage
+ html: String
+ styleSheet: String
+ scripts: String
+ display()
WebPageDecorator
page: WebPage
+ display()
AuthenticatedWebPage
+ authenticateUser()
AuthorizedWebPage
+ authorizedUser()

Step 1: Design the component interface

```
public interface WebPage {
     public void display();
}
```

Step 2: Implement the interface with your base concrete component class

```
public class BasicWebPage implements WebPage {
     private String html = ...;
     private String styleSheet = ...;
     private String scripts = ...;

     public void display() {
          /* Renders the HTML to the stylesheet, and run any
          embedded scripts */
          System.out.printIn("Basic web page");
     }
}
```

Step 3: Implement the interface with your abstract decorator class

```
public abstract class WebPageDecorator implements WebPage {
     protected WebPage page;

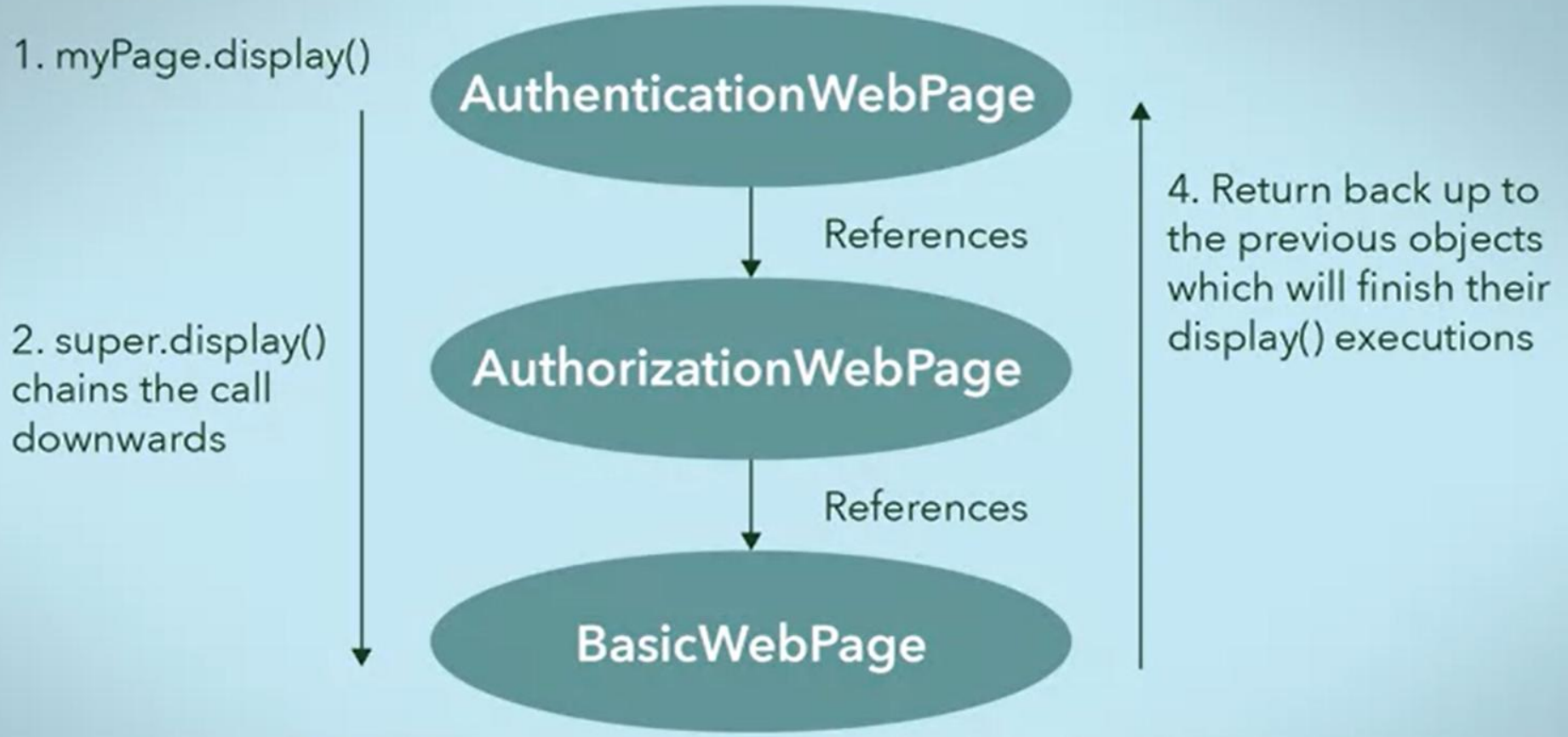
     public WebPageDecorator(WebPage webpage) {
          this.page = webpage;
     }

     public void display() {
          this.page.display();
     }
}
```

```
public class AuthorizedWebPage extends WebPageDecorator {
      public AuthorizedWebPage(WebPage decoratedPage) {
            super(decoratedPage);
      }
      public void authorizedUser() {
            System.out.println("Authorizing user");
      }
      public display() {
            super.display();
            this.authorizedUser();
      }
}

public class AuthenticatedWebPage extends WebPageDecorator {
      public AuthenticatedWebPage(WebPage decoratedPage) {
            super(decoratedPage);
      }
      public void authenticateUser() {
            System.out.println("Authenticating user");
      }
      public display() {
            super.display();
            this.authenticateUser();
      }
}
```

```
public class Program {
     public static void main(String args[]) {
          WebPage myPage = new BasicWebPage();
          myPage = new AuthorizedWebPage(myPage);
          myPage = new AuthenticatedWebPage(myPage);
          myPage.display();
     }
}
```


1. myPage.display()
AuthenticationWebPage
References
2. super.display() chains the call downwards
AuthorizationWebPage
References
BasicWebPage
4. Return back up to the previous objects which will finish their display() executions
3. The base concrete class does not aggregate any other class so it executes its display() behavior

WHAT ARE THE REASONS FOR USING THE DECORATOR DESIGN PATTERN?

The decorator design pattern allows objects to dynamically add behaviors to others.

To hide a complex, and resource demanding object until it is needed.

To reduce the number of classes needed to offer a combination of behaviors.

It lets you build a tree-like structure of objects that can be treated as a single, uniform object type.

WHAT IS THE DIFFERENCE BETWEEN PROXY AND DECORATOR DESIGN PATTERN?

- A protection proxy might be implemented like a decorator, but you should not forget the intent of a proxy.
- Decorators focus on adding responsibilities, but proxies focus on controlling the access to an object.
- Proxies differ from each other through their types and implementations.
- If you can remember their purposes, in most cases you will be able to clearly distinguish proxies from decorators

WHAT ARE THE KEY ADVANTAGES OF USING A DECORATOR



The existing structure is untouched, so you cannot introduce bugs there.



New functionalities can be added to an existing object easily.



You do not need to predict/implement all the supported functionalities at once (in other words, in the initial design phase). You can develop incrementally.

WHAT ARE THE DISADVANTAGES ASSOCIATED WITH THIS PATTERN?



Separate core and decorator features.



If you create too many decorators in the system, it will be hard to maintain and debug. So, in that case, they can create unnecessary confusion.



Handling conflicts between decorators.

REFLECTIVE QUESTIONS



When designing a system, what advantages does using the Decorator pattern provide compared to simply adding new functionality through inheritance?



How does the Decorator pattern support the Open/Closed Principle (open for extension, closed for modification)? Can you think of a scenario from your own coding experience where this would be useful?



What are some potential drawbacks of using multiple decorators in a system, and how might you balance flexibility with complexity?